PolNBDNet: Polarimetric Improvements, Step by Step

A working guide from the current 9-channel NBDNet to a publishable, general PolSAR despeckler. Each step has a plain-language explanation, technical detail with equations, and a completion check.

How to use this guide

Do the steps in order and change **one thing at a time**, evaluating on the same saved test set with the same seeds, and always next to a classical baseline (boxcar, refined Lee). It assumes the basic fixes are already done: per-channel normalization, overlapping-tile inference, a classical baseline, fixed seeds. Steps 1 to 6 run on the current subset; step 7 is the full-data run; steps 8 to 10 take it to paper level.

Step	What changes	Why it matters	Effort
1	Polarimetric evaluation suite	Shows whether polarimetry is preserved at all	Low
2	Valid covariance output (Cholesky)	Output is always a physically possible matrix	Low-medium
3	Polarimetric loss	Trains cross-channel structure, not just pixel values	Medium
4	Better target and realistic noise	Removes smoothing bias and simulation circularity	High
5	Condition on look count L	One model for all noise levels	Low
6	Multiscale architecture	Larger context, better texture	Medium
7	Full-data training protocol	Scale up once design is fixed	Server time
8	Multi-sensor and multi-mode support	Makes the model general	High
9	Real-data evaluation and downstream task	Evidence reviewers expect	Medium-high
10	Ablations and paper checklist	Shows which parts actually help	Medium

Notation

C is the 3×3 polarimetric covariance matrix of a pixel, built from the scattering vector $k = [S_{HH}, \sqrt{2}\, S_{HV}, S_{VV}]^T$. The 9 network channels are its 3 real diagonal entries plus the real and imaginary parts of its 3 complex upper off-diagonal entries. Z is the noisy L-look sample covariance, $\hat{C}$ is the model output and C_{ref} is the clean reference. $(\cdot)^H$ means conjugate transpose and q = 3 for quad-pol.

Step 1 · Polarimetric evaluation suite

In plain words

Right now we only check the total brightness image. That would not notice if the model destroyed the relationships between the radar channels, which is the whole point of polarimetry. This step adds the measurements that show what kind of scatterer each pixel looks like (surface, double-bounce, volume) and whether the model keeps that.

Technical detail

- Compute on every test tile: span, Pauli RGB, H/A/α, HH–VV coherence (magnitude and phase), Freeman–Durden (or Yamaguchi) components, ENL in homogeneous patches, ratio images.
- H/A/α come from the eigen-decomposition of the coherency matrix T (a fixed unitary transform of C, same eigenvalues). Report the error of $\hat{C}$ against C_{ref} for each parameter, not just their images.
- Validity check: report the fraction of output pixels whose minimum eigenvalue is ≥ 0 (a valid covariance matrix is positive semi-definite).
- Ratio image: noisy divided by denoised should look like pure speckle: mean ≈ 1, variance ≈ 1/L, and no visible scene structure. Structure left in it means the model removed real detail.

$$\text{Span} = C_{11} + C_{22} + C_{33} \qquad \gamma_{HH\text{-}VV} = |C_{13}| \,/\, \sqrt{(C_{11}\, C_{33})}$$

$$T = \Sigma_{i=1}^{3} \lambda_i\, u_i\, u_i^{H}, \quad p_i = \lambda_i \,/\, (\lambda_1{+}\lambda_2{+}\lambda_3)$$

$$H = -\,\Sigma\, p_i \log_3 p_i \qquad A = (\lambda_2 - \lambda_3) \,/\, (\lambda_2 + \lambda_3) \qquad \bar{\alpha} = \Sigma\, p_i\, \alpha_i$$

$$\text{ENL} = (\text{mean } I)^2 \,/\, \text{var}(I) \qquad r = I_{noisy} \,/\, I_{denoised}, \quad E[r] \approx 1,\ \text{var}(r) \approx 1/L$$

Done when

Run the suite on three things (noisy input, boxcar/Lee baseline, current NBDNet) and the table shows polarimetric numbers, not only PSNR. This becomes the headline table of the paper.

Step 2 · Valid covariance output (Cholesky layer)

In plain words

A covariance matrix is only physically possible if it has certain mathematical properties. The current network can output 9 numbers that do not make a valid matrix. The fix is to build the matrix from a simpler piece that always works: predict the matrix's 'square root' and multiply it by itself.

Technical detail

- Make the last layer predict a lower-triangular Cholesky factor L instead of C directly. The 9 real outputs are exactly enough: 3 real diagonal entries plus 3 complex lower entries (6 real numbers).
- Force the diagonal positive with softplus plus a small ε. Then $\hat{C} = L\, L^{H}$ is Hermitian and positive definite by construction.
- Compute the loss on the reconstructed $\hat{C}$ (convert diagonals to dB for scaling). Keep the log-determinant cheap: $\ln|\hat{C}| = 2\,\Sigma\, \ln l_{ii}$.
- Alternative: predict a Hermitian matrix S and output $\hat{C} = \exp(S)$ (log-Euclidean). It is also always valid, but needs eigendecompositions, which are slower and less stable on GPU.

$$L = [[l_{11}, 0, 0], [l_{21}, l_{22}, 0], [l_{31}, l_{32}, l_{33}]], \quad l_{ii} = \text{softplus}(z_{ii}) + \varepsilon$$

$$\hat{C} = L\, L^{H} \;\Rightarrow\; \hat{C}_{11} = l_{11}^{2}, \quad \hat{C}_{21} = l_{21}\, l_{11}, \quad \hat{C}_{31} = l_{31}\, l_{11}$$

$$\hat{C}_{22} = |l_{21}|^2 + l_{22}^{2}, \quad \hat{C}_{32} = l_{31}\, l_{21}^{*} + l_{32}\, l_{22}, \quad \hat{C}_{33} = |l_{31}|^2 + |l_{32}|^2 + l_{33}^{2}$$

Done when

The validity fraction from step 1 is 100% on all test tiles, and the quality metrics have not dropped compared with the previous model.

Step 3 · Polarimetric loss

In plain words

Plain 'average squared error' treats all 9 numbers as unrelated pixels and cares little about the delicate cross-channel values. We replace it with a score designed for radar covariance matrices, plus small extra penalties for the things we care about: brightness, channel correlation and scattering type.

Technical detail

- Main term: the Wishart (log-determinant) divergence between the clean reference and the prediction. It is zero only when they match and it respects matrix geometry.
- Alternative term: the affine-invariant Riemannian distance between $\hat{C}$ and C_{ref}; it uses the eigenvalues of $\hat{C}^{-1}C_{ref}$.
- Auxiliary terms: span error in dB, HH–VV coherence error (magnitude and phase) and H/α error computed from $\hat{C}$, so the model is rewarded for keeping scattering mechanisms.
- Introduce in stages: start from MSE, add the divergence, then one auxiliary term at a time, watching the step 1 metrics. Keep the weights small at first and ramp them up. Add εI before any inverse or log-determinant for stability.

$$D_W(C_{ref} \,\|\, \hat{C}) = \mathrm{tr}(\hat{C}^{-1} C_{ref}) - \ln |\hat{C}^{-1} C_{ref}| - q \quad (\geq 0, = 0 \text{ iff equal})$$

$$\text{Wishart NLL for noisy input:} \quad J = L\,[\, \ln|\Sigma| + \mathrm{tr}(\Sigma^{-1} Z)\,] \quad \text{(up to terms independent of } \Sigma)$$

$$d_R(\hat{C}, C_{ref}) = \sqrt{\Sigma_i \ln^2 \lambda_i(\hat{C}^{-1} C_{ref})}$$

$$J = J_{MSE} + w_1 D_W + w_2 J_{span} + w_3 J_{coh} + w_4 J_{H\alpha}$$

Done when

Off-diagonal and coherence errors from step 1 drop relative to the MSE-only model, with no loss in span quality.

Step 4 · Better target and realistic noise

In plain words

Our 'clean' image is itself a heavily smoothed average, so the model learns that smooth is correct and loses texture. Also, the fake noise we add is simpler than real radar noise. Here we fix both: use a better clean target, and make the noise behave like the real thing, or avoid fake noise completely by learning from real noisy pairs.

Technical detail

- **Multi-temporal target:** average co-registered repeat passes of the same site. For N independent passes of L looks each, the effective looks are about N·L, and texture is preserved because the averaging is over time instead of space. Mask pixels that changed between passes.
- **Realistic noise:** real SLC speckle is spatially correlated (oversampling, windowing) and forests are heavy-tailed. Filter white complex noise with a kernel fitted from a homogeneous patch, and add texture with the product model (Gamma texture gives K-distributed data; inverse-Gamma gives G0).
- **Self-supervised alternative:** split the SLC spectrum into two non-overlapping sub-looks Z_1, Z_2. They share the same underlying Σ but have independent speckle, so train $f(Z_1)$ to predict Z_2 (Noise2Noise). No simulation and no smoothed reference are needed.

- Decide the main route early. Multi-temporal needs careful co-registration; split-look is simpler but trades resolution for independence.

$$\text{Simulated speckle:}\quad k(x) = \Sigma^{1/2}\,(h * n)(x),\quad Z = (1/L)\,\Sigma_{l=1}^{L}\, k_l\, k_l^{H} \;\sim\; W(L, \Sigma/L)$$

$$\text{Product model:}\quad \Sigma(x) = \tau(x)\,\Sigma_0,\quad \tau \sim \text{Gamma}(\alpha, 1/\alpha)\quad (K),\quad \tau \sim \text{InvGamma}\quad (G0)$$

$$\text{Multi-temporal target:}\quad C_{\text{ref}} = (1/N)\,\Sigma_{k=1}^{N}\, C_k,\quad \text{ENL}_{\text{ref}} \approx N{\cdot}L$$

$$\text{Split-look:}\quad E[Z_2 \mid \Sigma] = \Sigma \;\Rightarrow\; \text{minimizing } J(f(Z_1), Z_2) \text{ gives } f(Z_1) = E[\Sigma \mid Z_1]$$

Done when

Retrained model keeps visible texture (stripes, edges) on test tiles, and the ratio image from step 1 shows no scene structure.

Step 5 · Condition on the look count

In plain words

The model currently does not know how noisy the input is, so it uses one setting for everything. That is why it does slightly worse on clean-ish inputs (L = 8). We simply tell it the noise level so it can adjust how hard it smooths.

Technical detail

- Feed log L to the network. Simplest: append a constant extra input channel equal to log L. Better: FiLM, where a small MLP maps log L to a per-channel scale and shift applied inside each block.
- For real data where L is not known exactly, estimate it from a homogeneous patch (ENL) or pass the sensor's nominal looks.
- Train with a wide range of L (for example 1 to 16) so the conditioning is actually learned.

$$h' = \gamma(L) \odot h + \beta(L),\quad [\gamma, \beta] = \text{MLP}(\log L)$$

Done when

At L = 8 the denoised CC is no longer below the noisy CC, and results at L = 1, 2, 4, 8 come from one model.

Step 6 · Multiscale architecture

In plain words

The current network looks at each pixel with a very small neighborhood, like reading a page through a keyhole. A network that also looks at a shrunken, wider view can tell what is a field, a forest edge or a building, and keeps texture better.

Technical detail

- Replace the flat stack with a U-Net style encoder–decoder: 3 downsampling stages, skip connections, and your Squeeze-Excitation/CBAM blocks inside. Alternatively a Swin-style transformer backbone.
- Keep the Cholesky output head (step 2) and the conditioning (step 5) at the decoder. Use reflect padding and overlapping tiles at inference.
- Grow the receptive field deliberately: with d downsampling stages and kernel k, it is roughly $(k-1)\cdot 2^d$ pixels per level, so 3 stages give a much wider view than the current network.
- Check parameter count and speed; a stronger model is only worth it if the step 1 metrics improve.

$$\text{Receptive field} \approx 1 + \Sigma_j\,(k_j - 1)\cdot \Pi_{i<j}\, s_i\ (s = \text{stride of earlier layers})$$

Done when

Texture and edges improve on test tiles, and the ablation table (step 10) shows a gain over the flat network.

Step 7 · Full-data training protocol

In plain words

Once steps 1 to 6 are settled on the small subset, we spend the server time on one carefully set up big run. No more experimenting on the big run, because it is expensive.

Technical detail

- Freeze architecture, loss, normalization and seeds. Scale the data by adding scenes (not just more tiles from one scene): more sites, more land covers, keeping scene-level train/val/test splits.
- Fit normalization on the training set only. Log polarimetric validation metrics every epoch, and restore best weights by a polarimetric metric as well as loss.
- Use mixed precision if available, regular checkpoints, and 2 to 3 seeds for the final model. Evaluate once on held-out scenes at L = 1, 2, 4, 8.
- Rough data target for a general claim: 15 to 30 scenes, several sensors/bands and land covers, about 10^5 to 5×10^5 tiles of 64×64.

$$\text{Reported result} = \text{mean} \pm \text{std over seeds, per land cover and per L}$$

Done when

A single table with the final model, baselines and subset results, all on the same saved test scenes.

Step 8 · Multi-sensor and multi-mode support

In plain words

A 'universal' model should work on different satellites and different polarization modes, not just one airborne sensor. That needs a common way of describing the data so the same network can read all of them.

Technical detail

- **Harmonize sensors:** calibrate to σ^0 or γ^0 using sensor metadata, fix a common channel order, and normalize each sensor with its own percentile range, fit on training data only.
- **Different modes:** quad-pol uses a 3×3 matrix (9 real channels), dual-pol a 2×2 matrix (4 real channels), compact-pol similar. Use a shared backbone with a mode-specific input stem and a Cholesky head of size q = 2 or 3, plus a mask for missing channels.
- Condition on incidence angle and band (L, C, X) the same way as on L in step 5.
- **Cross-sensor test:** train on one sensor and test on another, with and without fine-tuning, and report the drop honestly. Suggested mix: UAVSAR and NISAR (L-band quad-pol), ALOS-2 or RADARSAT-2 (quad-pol), Sentinel-1 (dual-pol).

$$\text{dual-pol:}\quad C \in \mathbb{C}^{2\times2},\quad \hat{C} = \tilde{L}\,\tilde{L}^{H}\quad \text{with } \tilde{L} \text{ lower-triangular } 2\times2 \text{ (4 real outputs)}$$

$$\text{conditioning vector:}\quad c = [\log L,\ \theta_{inc},\ \text{band},\ \text{mode}]\ \rightarrow \text{FiLM}$$

Done when

One set of weights runs on quad-pol and dual-pol data, and a cross-sensor result table exists.

Step 9 · Real-data evaluation and a downstream task

In plain words

Papers based only on added fake noise are criticized, because the answer key is made by the same process. Reviewers want to see the model working on real noisy images, and to see that cleaning actually helps a real task such as mapping land cover.

Technical detail

- **Real data, no simulation:** measure ENL in homogeneous patches (should rise), ratio-image statistics (mean ≈ 1, structure-free), edge preservation, point-target preservation (peak ratio in cities), and consistency of H/α histograms before and after.
- **Edge preservation:** compare the Laplacian (high-pass) of the denoised and noisy images; a value near 1 means edges are kept.
- **Downstream task:** run a fixed classifier (for example Wishart classifier or a random forest on decomposition features) on noisy, baseline-filtered and model-filtered data; report overall accuracy and Cohen's κ.
- **Failure analysis:** show tiles where the model loses to a baseline (cities, point targets) and explain why.

$$\mathrm{EPI} = \Sigma\,(\Delta I_d - \mathrm{mean})(\Delta I_n - \mathrm{mean}) \,/\, \sqrt{(\,\Sigma(\Delta I_d - \mathrm{mean})^2\,\Sigma(\Delta I_n - \mathrm{mean})^2\,)} \quad (\Delta = \text{Laplacian})$$

$$\text{peak ratio} = \max(I_d) \,/\, \max(I_n) \quad \text{around known point targets}$$

$$\kappa = (p_o - p_e) \,/\, (1 - p_e)$$

Done when

A real-data table and a downstream-task table show the model is better than the best classical filter, with an honest failure section.

Step 10 · Ablations and the paper checklist

In plain words

To claim that each idea helps, we switch the ideas on one at a time and show the effect. This is also the cleanest way to decide which ideas deserve to be in the paper.

Technical detail

- Run the ladder below on the same test set with 2 to 3 seeds each; report mean ± std and a paired significance test against the previous rung.
- Pick one main claim for the paper (for example: valid-covariance, polarimetry-preserving despeckler that generalizes across sensors) and keep the rest as supporting ablations.
- Compare against: boxcar, refined Lee, IDAN, PolSAR-BM3D or MuLoG, NBDNet on span, and at least one published deep PolSAR despeckler.
- Suggested route: conference paper after the full-data run (step 7), journal paper after steps 8 and 9.

$$\text{A0 baseline} \rightarrow \text{A1 + fixes} \rightarrow \text{A2 + Cholesky} \rightarrow \text{A3 + polarimetric loss} \rightarrow \text{A4 + L-conditioning}$$
$$\rightarrow \text{A5 + better target/noise} \rightarrow \text{A6 + multiscale} \rightarrow \text{A7 + multi-sensor}$$

Done when

A complete ablation table, baseline comparison, real-data results, downstream task and cross-sensor results, with one clear novelty claim.

Ablation ladder

Run	Added component	Main question
A0	Current model	Reference point
A1	Basic fixes (normalization, overlap tiles)	Do artifacts disappear?
A2	Cholesky output	Always valid, quality kept?
A3	Polarimetric loss	Better off-diagonals and coherence?
A4	L-conditioning	L = 8 degradation fixed?
A5	Better target and noise	Texture preserved?
A6	Multiscale backbone	Does context help?
A7	Multi-sensor, multi-mode	Generalizes across sensors?

Measure polarimetry properly, make the output a valid matrix, teach the model what polarimetric error means, give it a better answer key and realistic noise, tell it the noise level, let it see more context, then scale up and prove it on real data from different satellites.